

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

Выполнил:

Попов Дмитрий

Группа

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача - настроить сетевое взаимодействие между сервисами.

Ход работы

Для достижения поставленных задач были выполнены следующие шаги по контейнеризации и оркестрации микросервисного веб-приложения:

1. Разработка Dockerfile для каждого микросервиса

Для каждого микросервиса (API Gateway, User Service, Nutrition Service, Workout Service, Fasting Service, Progress Service) был разработан отдельный Dockerfile. Структура файлов идентична, за исключением портов, которые каждый сервис использует.

Пример Dockerfile для api-gateway:

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
RUN npm run build
```

```
EXPOSE 3000
```

2. CMD ["node", "dist/index.js"]

Здесь мы определяем базовый образ, устанавливаем рабочую директорию, копируем зависимости, устанавливаем их, копируем исходный код, собираем приложение, указываем порт и команду запуска.

3. Разработка общего docker-compose.yml

Файл `docker-compose.yml` был создан для определения и запуска многоконтейнерного приложения. Он описывает все сервисы, их образы, переменные окружения, порты, тома и зависимости.

○ Сервис базы данных (db):

Используется образ `postgres:16.9-alpine`. Настроен автоматический перезапуск. Учётные данные передаются через переменные окружения. Порт 5432 мапируется на внешний порт 5433. Для сохранения данных используется том `db_data`. При первом запуске выполняется SQL-скрипт инициализации из

директории `/db-init`, создающий отдельные базы данных для каждого микросервиса. Добавлен `healthcheck` для проверки готовности БД.

- Микросервисы:

Каждый микросервис собирается из своего `Dockerfile`. Порты мапятся для доступа. Каждый сервис использует свою базу данных внутри одного экземпляра PostgreSQL.

- API Gateway:

Выступает единой точкой входа. Собирается из своего `Dockerfile`, порт мапируется.

- Зависимости сервисов (`depends_on`):

Все микросервисы (кроме `api-gateway`) зависят от сервиса `db` с условием `condition: service_healthy`.

Сервис `progress-service` зависит от `user-service`, `nutrition-service`, `workout-service` и `fasting-service` с условием `condition: service_started`.

`api-gateway` зависит от всех остальных микросервисов с условием `condition: service_started`.

4. Разработка общего `.env` файла и передача переменных окружения

В корне проекта создан файл `.env`, содержащий все необходимые переменные окружения: учётные данные БД, порты сервисов, секретный ключ JWT. Переменные автоматически подгружаются Docker Compose и передаются в каждый сервис. Внутри микросервисов они читаются через `process.env` с использованием библиотеки `dotenv`.

Вывод

В рамках лабораторной работы было настроено сетевое взаимодействие между сервисами веб-приложения FRESH. Каждый микросервис был контейнеризован с помощью индивидуального `Dockerfile`. Общий файл `docker-compose.yml` обеспечил оркестрацию всех компонентов, включая базу данных, с правильным порядком запуска и управлением зависимостями. Использование `healthcheck` и томов для данных гарантирует отказоустойчивость и сохранность информации. Все сервисы взаимодействуют через API Gateway, что обеспечивает масштабируемость и поддерживаемость системы.